CS780 Capstone Project:

OBELIX: the Warehouse Robot

The best way to learn Reinforcement Learning is to get your hands dirty by actually doing it.

Table of Contents

Table of Contents **2**
1. Your First Job in the Industry 3
2. Task Description 4
    2.1 Setting 4
    2.2 Difficulty Levels 5
    2.3 Competition Format 6
4. Environment and Codebase 7
    4.1 Environment Description 7
5. Evaluation 9
    5.1 Metric 9
    5.2 Development/Test Phase Evaluation 9
    5.3 LLM Usage Policy 10
6. Getting Started: Implementation and Code 10
    6.1 How to get started? 11
    6.2 Starter Code 11
        6.2.1 Double Deep Q-Network (DDQN) 12
7. Codabench Leaderboard 12
    7.1 Competition Link 12
    7.2 Registration 13
    7.3 Submission Policy 13
    7.4 Environment & Dependency Requirements 13
8. Evaluation / Grading Criteria 14
9. Submission Details: Report and Code 14
10. Timeline 15
    10.1 Phase 1 (Level 1 Dev Phase) 15
    10.2 Phase 2 (Level 2 Dev Phase) 15
    10.3 Phase 3 (Level 3 Dev Phase) 15
    10.4 Phase 4 (Test Phase) 15
11. Possible Ideas 15
    11.1 Background Tutorials 15
    11.2 Tabular / Value-based Methods 16
    11.3 Function Approximation and Deep RL 16
    11.4 Evolutionary Strategies & Heuristics 16
    11.5 What approach should I take? 16
12. Lost? Whom to contact?
13. Challenges 17
14. References 17

NOTE: This is a dynamic document, it will keep getting updated, please keep checking it regularly.

1. Your First Job in the Industry

You have recently graduated and joined a robotics startup that develops warehouse robots for large e-commerce companies. The company has been building delivery and warehouse robots that need to interact (e.g., carry, move, etc.) with objects in dynamic environments. The CTO wants to move beyond hand-coded behaviors (which are brittle and hard to scale) and leverage modern reinforcement learning to make robots learn complex manipulation tasks from trial-and-error feedback. Your task is to design an RL-based policy that maps sensor observations to robot actions.

CTO got to know that you have done CS780: Deep Reinforcement Learning course from IITK; she decided to hand over this project to you since you are among the few ones (due to CS780 background) who know DRL inside out. The CTO assigns you the task of developing an autonomous controller for a mobile robot (called as **OBELIX**) that can locate, attach to, and push objects (like boxes) to desired regions reliably even in partially observable or changing conditions.

Meanwhile, someone else in the company also claims that they can hand-craft or evolve a better controller. To make it fair and exciting, the CTO organizes an internal competition: design the best algorithm for the **OBELIX** robot in a simulated arena. The robot draws inspiration from the classic 1991 paper by Sridhar Mahadevan and Jonathan Connell [1], one of the earliest real demonstrations of reinforcement learning on a physical robot.

In this competition, you will implement a RL algorithm that takes binary or discrete sensor readings as input and outputs one of five discrete actions. The goal is to make the robot find the box, attach/push it, and drive it to the boundary (goal) as quickly and reliably as possible across randomized starting positions and increasing difficulty levels.

This is an **individual competition**, not a group project. There will be a public leaderboard on Codabench where your solution will be ranked. **Do not share code or models with classmates** unless you want to share grades and career too! Code similarity checks and viva will be strict : copying leads to **F grade, no questions asked**!

Everyone must participate actively: aim for multiple submissions during the development phase to show your experimentation efforts.

2. Task Description

2.1 Setting

You are given a robot equipped with discrete sensors and a fixed set of 5 actions: rotate right 45°, rotate right 22°, move forward, rotate left 22°, and rotate left 45° (details mentioned in section 4). The objective is to design an algorithm (policy) that finds, pushes and unwedges (at the goal state) the grey box from the arena while using the minimum possible number of steps within an episode.

At the start of each episode, a grey box is generated at a random location inside the arena. To remove the box, the robot must navigate toward it using its sensor observations, attach to it, and push it until the particle reaches the arena boundary, which defines the goal region. The robot automatically attaches to the grey box upon contact and automatically unwedges/detaches once the box reaches the boundary. The box is considered removed only after it is successfully pushed outside the boundary.

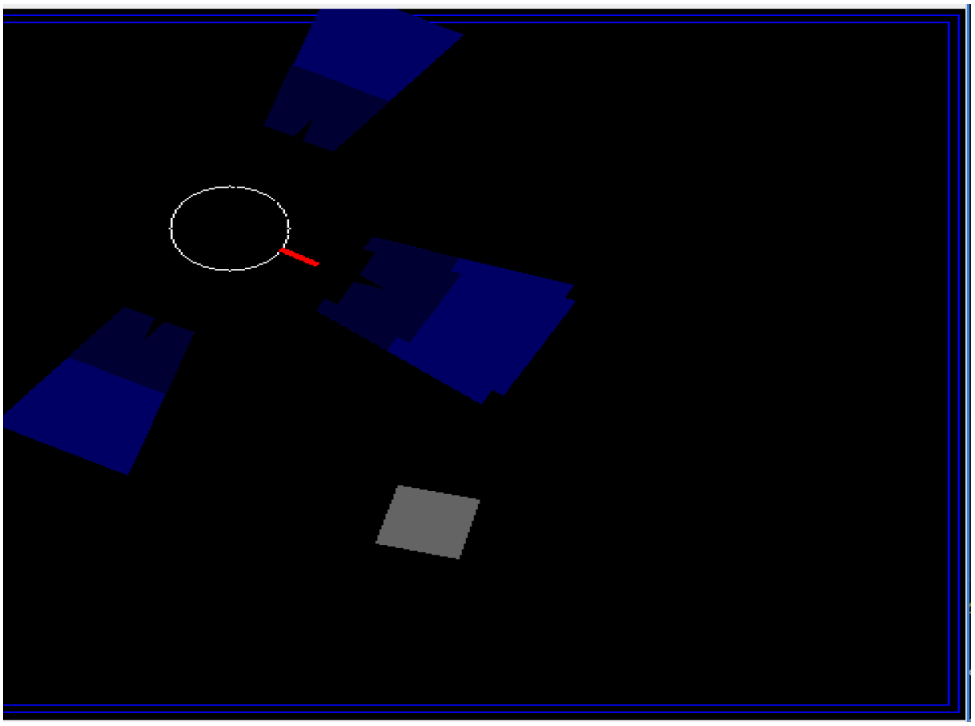
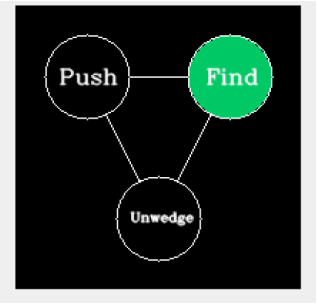


Image of the robot and environment. The sensors and their range is visible along with the infrared front sensor. The blue boundary is the goal state.

NOTE: This is a dynamic document, it will keep getting updated, please keep checking it regularly.

The robot observes the environment using sonar-like sensors that indicate whether the box is near or far. The figure above shows the robot, arena, grey box, boundary area sensor range, and directions. There are a total of 8 sensors around the robot, 4 towards the front and 2 on either side. The sensors provide a bit-based output that is set to 1 if the sensor detects the box within its region. If the box is in the light blue area, the near bit is set to 1; if it's in the dark blue area, the far bit is set to 1. The robot also has an infrared sensor (IR sensor) (represented as a red line on the front), which is set to 1 if the grey box is sensed by it. The robot receives an 18-bit observation vector consisting of 16 sonar bits (near + far), 1 infrared sensor bit, and 1 bit that indicates whether the robot is stuck against a wall or boundary.

For human readability, these are mentioned as 3 distinct states. First, the robot “Finds” the grey box, then attaches to it and “Pushes” the box, and finally “Unwedeges” the box at the boundary. Note, these states are **conceptual for humans** and are **not directly given to the robot**. The robot must infer them from sensor readings.



Discrete states describing the reward behaviour. This is not the actual state visible to the agent

Note that, for the robot, the environment is partially observable, since it only sees within the range allowed by its sensors. If the box is far from the robot, it may not even know exactly where the box is. It would first need to explore a bit to get within the box's range. More specifically, the robot does not know the full state of the environment. It only receives **local sensor readings (observations)**, making this a **Partially Observable Markov Decision Process (POMDP)**.

The agent is evaluated based on the cumulative reward obtained at the end of the episode. At each step, the robot receives a positive reward if any sensor bit is set to 1 and it moves forward. Details on the reward obtained at each step are mentioned in section on Environment and Codebase details. Episodes terminate when either the box is unwedged or a maximum step limit (episode horizon) is reached.

2.2 Difficulty Levels

There are 3 difficulty levels across which the robot is evaluated.

Level 1 – Static Box

- Box remains stationary

- Robot must locate and push it to boundary

Level 2 – Blinking Box

- Box randomly **appears and disappears**
- Robot **cannot attach when box is invisible**
- Once attached → box stops blinking

Level 3 – Moving + Blinking Box

- Box moves with constant velocity
- Box also blinks (like Level 2)
- Movement stops once robot attaches

All these settings increase the difficulty by introducing partial observability and non-stationary dynamics.

Additionally, in each of the levels, a **vertical wall with a small opening** may appear.

- The robot cannot push through the wall.
- If the robot pushes the box into the wall, it receives a negative **reward**.
- The robot must navigate through the opening to reach the boundary.

2.3 Competition Format

There will be essentially two phases of the competition: Development Phase and Testing Phase.

Further, the development phase is sub-divided into three sub-phases, each corresponding to three levels above. The final test phase will have combinations of all the three levels.

For each (sub-)phase you will have to submit the learned RL policy algorithm (in the form of a python code file) on Codabench (details provided later in the document). The trained agent with a fixed policy is evaluated on all three difficulty levels for 10 runs/episodes with fixed hidden seeds on Codabench. Cumulative reward is captured across each episode and averaged across 10 episodes to get the mean cumulative reward for each difficulty level. This is displayed on the leaderboard along with an

average reward score across all the difficulty levels. Note that the evaluation environment uses **hidden random seeds**, meaning you cannot overfit to specific scenarios.

Check out the Competition timeline in here

4. Environment and Codebase

The simulation is in this GitHub repo: https://github.com/Exploration-Lab/CS780-OBELIX

```
git clone https://github.com/Exploration-Lab/CS780-OBELIX
cd CS780-OBELIX
pip install -r requirements.txt
```

Please use the instructions provided in the README.md file present in the repo.

- Run `manual_play.py` to drive the robot manually (keys: w=forward, a/q=left turns, d/e=right turns) and observe sensor feedback and reward in the terminal and the states in another box (as shown in the images above). Note you can run the environment on your laptop (CPU), GPU is not required.

4.1 Environment Description

- **Observations (partial state):** A vector of 18-bit readings (16 bits from the sonar sensors, a bit for the infrared sensor, and a stuck bit)
  - Example observation:

```
obs = [0,1,0,0,1,0,0,1, ...]  # length 18
```

Sensor Type	Count	Description
Sonar sensors (**near**)	8	Detect box in **near** region
Sonar sensors (**far**)	8	Detect box in **far** region

Infrared sensor	1	Detect box directly ahead
Bit 17	1	Set to 1 when robot is stuck against wall/boundary (stuck_flag). Attachment state is not visible in observations.

- **Actions:** Discrete 5: "L45" (left 45°), "L22" (left 22.5°), "FW" (forward), "R22", "R45".

Action	Description
L45	Rotate left 45°
L22	Rotate left 22.5°
FW	Move forward
R22	Rotate right 22.5°
R45	Rotate right 45°

- **Rewards:** The agent receives a reward at each time step according to the observation table below

Observation	Reward
left/right far/near sensor bit is set to 1 (this reward is incurred only once during an episode)	+1
Forward sensor **far** bit is set to 1 (this reward is incurred only once during an episode)	+2
Forward sensor **near** bit is set to 1 (this reward is incurred	+3

only once during an episode)	
Infrared sensor bit is set to 1 (this reward is incurred only once during an episode)	+5
• Robot is stuck against wall • Robot is pushing grey box into wall • Robot is not able to move forward at the arena boundary without a grey box	-200
None of the sensor bits are set to 1	-1
On enabling “push” state. Initial attachment to grey box	+100
Each step in “push” state	-1
Grey box at boundary (Successful termination)	+2000

- **Difficulty levels** (via flags in `evaluate.py`):
  - Static box
  - Blinking/appearing-disappearing box
  - Moving + blinking box (adjust --box_speed N)
  - Optional: --wall_obstacles for enabling the wall obstacle

- **Scoring:** Cumulative reward over fixed max_steps (2000 steps); terminal bonus on success; mean/std over multiple runs/seeds.

You need to implement policy algorithm in python and store it in file named as: `agent_template.py`, the file should contain the following function with the definition given below (it may have other helper functions), more details in section on Submission Details:

```
def policy(obs) -> str:
# return one of: "L45", "L22", "FW", "R22", "R45"
```

In above “obs” is the input observations

Note: Current push is "attach and stick" (simplified); real physics pushing not fully simulated.